University of Exeter

ECM2433: The C Family

Lecture 09: Lifetimes

The problem: dangling references

```
1 fn main() {
2     let r;
3     {
4         let x = 5;
5         r = &x;
6     } // x is dropped here
7     println!("{r}"); // r is now dangling
8 }
```

- `r` borrows `x`, but `x` goes out of scope on line 6
- After the inner block, `r` points to invalid memory
- In C, a dangling pointer is undefined behaviour: it may appear to work, crash, or corrupt memory
- Rust rejects this at compile time
- [Book ch 10.3]

University of Exeter

How the borrow checker compares scopes

```rust
fn main() {
    let r;                  // ---------+-- 'a
    {                       //          |
        let x = 5;          // -+-- 'b  |
        r = &x;             //  |       |
    }                       // -+       |
    println!("{r}");        //          |
}                           // ---------+
```

- The borrow checker compares the lifetime of the reference (`'a`) with the data lifetime (`'b`)
- `'b` is shorter than `'a`: the data is gone while the reference still exists
- Rust rejects the code because the reference outlives the data
- [Book ch 10.3]

University of Exeter

The problem in functions

```rust
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
```

```
[gad212@mcrugcomp01 hello_rust]$ cargo run
   Compiling hello_rust v0.1.0 (/home/links/gad212/hello_rust)
error[E0106]: missing lifetime specifier
 --> src/main.rs:1:33
  |
1 | fn longest(x: &str, y: &str) -> &str {
  |               ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the
signature does not say whether it is borrowed from `x` or `y`
```

- The function returns a reference, but the compiler does not know which input it comes from
- Could be `x` or `y`, so the compiler cannot verify the return value lives long enough
- We need to tell the compiler how the input and output lifetimes relate
- [Book ch 10.3]

University of Exeter

Lifetime annotation syntax

```
&i32          // a reference
&'a i32       // a reference with lifetime 'a
&'a mut i32 // a mutable reference with lifetime 'a
```

- Lifetime parameters start with `'` and are usually short: `'a`, `'b`, `'c`
- Placed between the `&` and the type
- Annotations describe relationships between references; they do not change how long they live
- A single annotation on its own is meaningless: lifetimes only matter when they connect inputs to outputs
- [Book ch 10.3]

University of Exeter

The longest function with lifetimes

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("long string");
    let s2 = "xy";
    let result = longest(s1.as_str(), s2);
    println!("The longest string is: {result}");
}
```

- `<'a>` after the function name declares a lifetime parameter, like `<T>` declares a type parameter
- Both inputs and the output share `'a`: the return value cannot outlive either input
- The compiler uses `'a` to verify the returned reference is valid wherever it is used
- [Book ch 10.3]

University of Exeter

Thinking in terms of lifetimes

```rust
// Only returns x, so only x needs the lifetime
fn first<'a>(x: &'a str, y: &str) -> &'a str {
    x
}

// Does NOT compile: cannot return a reference to local data
fn broken<'a>(x: &'a str, y: &str) -> &'a str {
    let s = String::from("hello");
    s.as_str()
}
```

```
error[E0515]: cannot return value referencing local variable `s`
 --> src/main.rs:9:5
  |
9 |     s.as_str()
  |     -^^^^^^^^^
  |     |
  |     returns a value referencing data owned by the current function
  |     `s` is borrowed here
```

- If only one input can be returned, only that input needs the lifetime
- The return value's lifetime must always be tied to an input's lifetime
- You cannot return a reference to data created inside the function: it would be dropped when the function returns
- When a function creates new data, return an owned type like `String` instead
- [Book ch 10.3]

Lifetime elision rules

```rust
// We write this:
fn first_word(s: &str) -> &str { ... }

// The compiler sees this:
fn first_word<'a>(s: &'a str) -> &'a str { ... }
```

- The compiler applies three elision rules to infer lifetimes automatically:
- Rule 1: Each reference parameter gets its own lifetime (`'a`, `'b`, ...)
- Rule 2: If there is exactly one input lifetime, it is assigned to all output lifetimes
- Rule 3: If one parameter is `&self` or `&mut self`, its lifetime is assigned to all output lifetimes
- If the rules do not fully determine the output lifetimes, the compiler asks you to annotate
- [Book ch 10.3]

University of Exeter

Reading lifetime error messages

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("long string");
    let result;
    {
        let s2 = String::from("xy");
        result = longest(s1.as_str(), s2.as_str());
    } // s2 is dropped here
    println!("{result}"); // result might reference s2
}
```

```
[gad212@mcrugcomp01 hello_rust]$ cargo run
   Compiling hello_rust v0.1.0 (/home/links/gad212/hello_rust)
error[E0597]: `s2` does not live long enough
  --> src/main.rs:10:39
   |
 9 |         let s2 = String::from("xy");
   |             -- binding `s2` declared here
10 |         result = longest(s1.as_str(), s2.as_str());
   |                                       ^^ borrowed value does not live long enough
11 |     } // s2 is dropped here
   |     - `s2` dropped here while still borrowed
12 |     println!("{result}"); // result might reference s2
   |                -------- borrow later used here
```

- The lifetime annotations on `longest` mean the result cannot outlive either input
- `s2` is dropped before `result` is used, so the compiler rejects the call
- The error message points to three locations: the borrow, the drop, and the later use
- [Book ch 10.3]

Combining lifetimes with generics

```rust
fn longest_with_message<'a, T: Display>(x: &'a str, y: &'a str, message: T) -> &'a str {
    println!("{message}");
    if x.len() > y.len() { x } else { y }
}
```

- Lifetime parameters and type parameters go in the same angle brackets: `<'a, T>`
- Lifetimes are listed first by convention
- Trait bounds work as before: `T: Display`
- The lifetime `'a` still ties the inputs to the output
- [Book ch 10.3]

University of Exeter

Demo

University of Exeter

Recap

- Lifetimes prevent dangling references: compiler ensures references never outlive what they point to
- Annotation syntax: `&'a T` ties references together; `<'a>` declares the lifetime parameter
- Lifetime annotations describe relationships; they do not change how long data lives
- Elision rules: the compiler infers lifetimes when there is one input reference, or `&self`
- When the compiler cannot infer, it asks you to annotate. Read the error message for guidance!
- Lifetimes combine with generics in the same angle brackets: `<'a, T>`
- Required reading: [Book ch 10.3]

University of Exeter